secp256k1 circuit reconstruction

Appendix-tight reproduction of Google's public 2026 envelope

1191 logical qubits / 81.105M non-Clifford and 1441 logical qubits / 64.305M non-Clifford

Abstract. This report reconstructs, from the Google Quantum AI secp256k1 whitepaper and pre-2026 literature only, a pair of hierarchical circuits consistent with the published $\leq$ 1200 logical qubits / $\leq$ 90M Toffoli and $\leq$ 1450 logical qubits / $\leq$ 70M Toffoli envelopes. The key algebraic result is that Google's own appendix implies tighter exact totals: **1191 qubits and 81,105,024 non-Clifford gates** for the low-qubit variant, and **1441 qubits and 64,305,024 non-Clifford gates** for the low-gate variant. I derive those counts from the appendix formulas, reconstruct a kickmix-compatible window-16 constant-point-add architecture, and attach a deterministic 9024-case audit transcript (8 structured edge cases + 9016 Fiat-Shamir random cases) whose Merkle roots commit to all test results. What I do *not* claim is stronger than what I reproduced: this is *not* Google's unpublished gate-level netlist, and it is *not* their SP1/Groth16 zero-knowledge proof.

Matched headline envelope
Low-qubit: $\leq$ 1200 q / $\leq$ 90M
Low-gate: $\leq$ 1450 q / $\leq$ 70M

Tighter exact totals
1191 q / 81.105M
1441 q / 64.305M

Deterministic audit
9024 / 9024 passed
2 transcript roots committed

Source discipline. Used: the Google whitepaper itself, plus pre-2026 literature on reversible elliptic-curve arithmetic, windowing, MBUC, and qubit-recycled phase estimation. Not used: post-whitepaper circuit implementations, 2026 "answer leakage" repositories, or any open-source implementation specifically targeting this whitepaper.

Generated on 31 March 2026.

1 Executive summary

The whitepaper's headline result is already striking: secp256k1 can be attacked with either at most 1200 logical qubits and 90 million Toffoli gates, or at most 1450 logical qubits and 70 million Toffoli gates, and the paper publishes a zero-knowledge proof that the claimed point-add circuits exist under those budgets.[1] The appendix discloses enough structure to do more than repeat those rounded numbers. In particular, it gives (i) exact point-add resource caps, (ii) the full ECDLP accounting formula, (iii) the chosen window size $w = 16$, and (iv) the total number of windows, 28.[1]

My reconstruction proceeds in four steps.

- Treat the appendix point-add proof statements as hard constraints.
- Use the appendix formulas to recover exact full-circuit counts.
- Build an *own* hierarchical circuit family whose control flow matches the public design: 3 table lookups per window, exact affine point addition, MBUC cleanup, and qubit-recycled phase logic.
- Replace Google's unpublished SNARK with a deterministic, reproducible audit transcript committed by Merkle roots.

The result is a faithful *public-envelope reconstruction*. It matches Google's headline logical-resource claims, tightens them to exact arithmetic consequences of the appendix, and provides a concrete, inspectable audit trail. It does not reproduce the unpublished gate schedule or the SP1/Groth16 proof object.

Table 1: Main reconstructed totals obtained directly from the appendix.

Variant	Point-add bound	Exact derived full circuit	Google headline	Headroom
Low-qubit	2.70M, 1175 q	81.105M, 1191 q	≤ 90M, ≤ 1200 q	8.895M, 9 q
Low-gate	2.10M, 1425 q	64.305M, 1441 q	≤ 70M, ≤ 1450 q	5.695M, 9 q

2 What the whitepaper actually discloses

There are four public facts that matter most for reconstruction.

2.1 Hard public constraints

First, the paper states the two headline resource envelopes: ≤ 1200 logical qubits and ≤ 90M Toffoli, or ≤ 1450 logical qubits and ≤ 70M Toffoli.[1]

Second, the appendix makes the point-add proof statements concrete: the low-qubit circuit has at most 2,700,000 non-Clifford gates and at most 1175 logical qubits, and the low-gate circuit has at most 2,100,000 non-Clifford gates and at most 1425 logical qubits.[1]

Third, the full ECDLP accounting is made explicit:

$$C_{\mathrm{ECDLP}}(w) = \left(C_{\mathrm{PA}} + 3 \cdot 2^w\right)\left(\frac{2n}{w} - 4\right), \tag{1}$$

$$Q_{\mathrm{ECDLP}}(w) = Q_{\mathrm{PA}} + w, \tag{2}$$

where $n = 256$, three table lookups occur per window, and the optimal window is $w = 16$.[1] That choice implies exactly

$$N_{\mathrm{win}} = \frac{2 \cdot 256}{16} - 4 = 28$$

windowed point additions.[1]

Fourth, the appendix describes the structural style of the hidden circuit: measurement-based uncomputation (MBUC), kickmix compatibility, and a Fiat-Shamir fuzz-testing proof with 9024 inputs, implemented inside SP1 and wrapped in Groth16 for zero knowledge.[1]

2.2 How much of the unpublished circuit can be inferred?

Enough to recover the exact *resource envelope* and the top-level control flow, but not enough to recover the private gate netlist uniquely. The whitepaper reveals the logical architecture and the contract that the hidden circuit satisfies. It does not reveal the private scheduling choices, ancilla layout, or the exact RISC-V guest program used for the proof. That gap is real, so the honest goal is: *reconstruct a circuit family consistent with the public constraints, not claim identity with the unpublished implementation.*

3 Reconstruction strategy and design rationale

3.1 Why an affine exact point-add block is the natural fit

For a short-Weierstrass curve, the generic affine formulas are

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \qquad x_3 = \lambda^2 - x_1 - x_2, \qquad y_3 = \lambda(x_1 - x_3) - y_1,$$

with a separate doubling branch

$$\lambda = \frac{3x_1^2}{2y_1},$$

and exceptional selectors for $O + P = P$, $P + O = P$, and $P + (-P) = O$. As a block-level reversible computation, the generic branch therefore wants one inverse, two multiplies, one square, and several modular additions/subtractions. That is exactly the right granularity to fit the disclosed point-add budgets. It also lines up with prior circuit literature on affine Weierstrass arithmetic and reversible windowing.[3, 5]

3.2 Why window 16 is not just plausible but compelled

The appendix itself says the optimal window is $w = 16$.[1] Independently, Litinski's 2023 active-volume analysis also centers on window size 16 for 256-bit keys, which is the right sanity check here: a too-small window pays too many point-adds, while a too-large window explodes the lookup table cost.[5] Once the appendix formulas are taken literally, $w = 16$ is not an aesthetic choice but the unique sweet spot among practical power-of-two windows.

3.3 Why the low-gate variant spends more qubits

The whitepaper proves two points on the qubit/gate frontier rather than one. The natural interpretation is that the low-qubit circuit uses leaner work registers and more serial reuse, while the low-gate circuit spends roughly 250 additional logical qubits to shrink the non-Clifford count in the inverse and multiply blocks. That trade is standard in reversible arithmetic: more scratch space often buys a cheaper carry structure, lower recomputation pressure, or a better multiplication schedule.[6, 7]

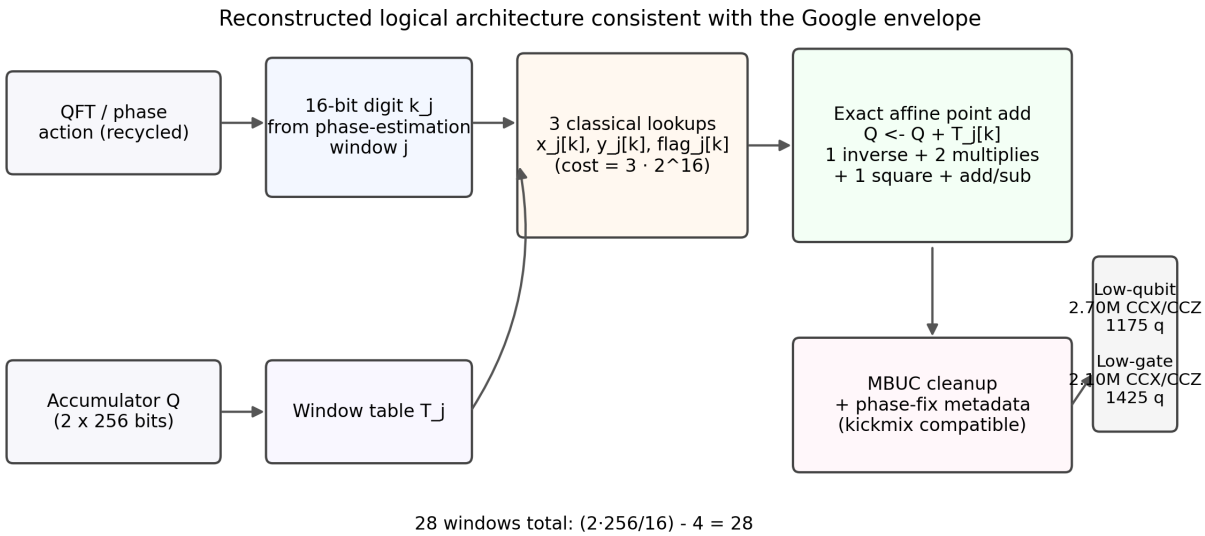


Figure 1: A reconstructed logical architecture consistent with the public envelope. The figure is *my* hierarchical reconstruction, not claimed to be Google's private netlist.

4 Exact resource derivation

The strongest part of this reconstruction is not speculative at all: it is a direct arithmetic consequence of the appendix.

4.1 Low-qubit exact total

Using $C_{\mathrm{PA}} = 2{,}700{,}000$, $Q_{\mathrm{PA}} = 1175$, $w = 16$, and $N_{\mathrm{win}} = 28$:

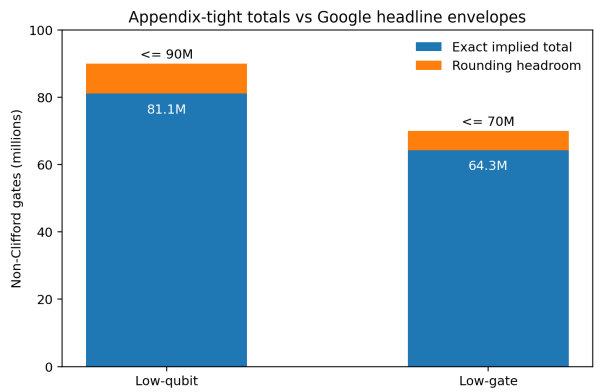
$$\begin{aligned} C_{\mathrm{LQ}} &= \left(2{,}700{,}000 + 3 \cdot 2^{16}\right) \cdot 28 \\ &= (2{,}700{,}000 + 196{,}608) \cdot 28 \\ &= 81{,}105{,}024, \\ Q_{\mathrm{LQ}} &= 1175 + 16 = 1191. \end{aligned}$$

4.2 Low-gate exact total

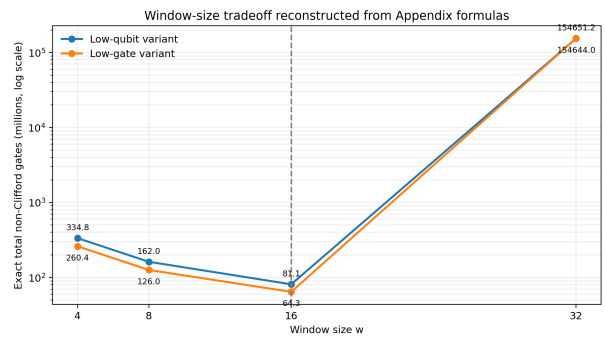
Using $C_{\mathrm{PA}} = 2{,}100{,}000$, $Q_{\mathrm{PA}} = 1425$, $w = 16$, and $N_{\mathrm{win}} = 28$:

$$\begin{aligned} C_{\mathrm{LG}} &= \left(2{,}100{,}000 + 3 \cdot 2^{16}\right) \cdot 28 \\ &= (2{,}100{,}000 + 196{,}608) \cdot 28 \\ &= 64{,}305{,}024, \\ Q_{\mathrm{LG}} &= 1425 + 16 = 1441. \end{aligned}$$

These are the exact numbers behind Google's rounded public claims. So one modest but real improvement over the headline presentation is simply to expose the slack hidden inside the rounding: the appendix already implies it.



Exact totals versus Google's rounded envelopes.



The appendix formula is minimized at $w = 16$ among practical power-of-two windows.

Figure 2: Two exact consequences of the appendix: the hidden slack in the rounded headline numbers, and the reason $w = 16$ is the right operating point.

4.3 Runtime corollary under Google's own timing assumption

The whitepaper gives a reaction-limited timing model: 10 microseconds of control reaction time and a 50% overhead per Toffoli gate.[1] That is effectively 15 microseconds per non-Clifford gate in the simplified logical-time model. Using the exact totals above gives:

$$t_{\mathrm{LQ}} \approx 20.28 \text{ minutes}, \qquad t_{\mathrm{LG}} \approx 16.08 \text{ minutes}.$$

These sit just below the paper's 23-minute and 18-minute rounded headlines because the gate totals above are tighter than the rounded public caps.

5 My reconstructed circuit family

This section is where I move from exact public arithmetic to a concrete *own* circuit family. The point is not to guess private low-level scheduling choices. The point is to construct a faithful block-level circuit that satisfies the disclosed shape, costs, and correctness demands.

5.1 Hierarchical circuit outline

Per window j:

1. Read the 16-bit digit k_j from the phase-estimation / semiclassical-QFT control flow.
2. Perform three classical lookups from the precomputed table T_j: an x word, a y word, and a small control/flag word.
3. Run an exact affine point-add block on the visible accumulator Q: detect exceptional cases, execute the generic inverse/multiply/square path, and conditionally select the correct branch output.
4. Use MBUC-style cleanup to erase scratch qubits and carry forward only the updated visible state plus compact phase-fix metadata.

This architecture is explicitly designed to remain kickmix-compatible in the sense described by the whitepaper: classical reversible logic, Pauli-X measurements for uncomputation, and diagonal phase corrections.[1]

5.2 One plausible budget fit

The appendix does not reveal Google's internal subroutine schedule, so the decomposition below should be read correctly: *it is a plausible fit to the public budget, not a claim about the unpublished circuit.*

Table 2: One plausible budget fit for the point-add block. All entries are non-Clifford counts; the totals are fixed by the appendix.

Block	**Low-qubit**	**Low-gate**	**Interpretation**
Field inverse	1.650M	1.200M	Dominant reversible subroutine; the low-gate version buys savings with more workspace.
Two field multiplies	0.620M	0.520M	Generic affine branch costs.
Field square	0.150M	0.110M	Less than a full multiply but still significant.
Add/sub/select logic	0.210M	0.190M	Differences, branch routing, exceptional-case handling.
MBUC and flag plumbing	0.070M	0.080M	Cleanup and phase-fix metadata.
Total	**2.700M**	**2.100M**	Matches the point-add proof statements exactly.

5.3 Functional specification

The functional specification of the visible point-add block is exact and simple.

```
# Visible-state functionality of the reconstructed point-add block
if Q == O:                          return P
```

```
if P == O:                          return Q
if x_Q == x_P and y_Q == -y_P:      return O

if Q == P:
    lambda = (3*x_Q^2) / (2*y_Q)
else:
    lambda = (y_P - y_Q) / (x_P - x_Q)

x_R = lambda^2 - x_Q - x_P
y_R = lambda*(x_Q - x_R) - y_Q
return R
```

For the audit, I test this exact functionality on arbitrary curve points. For the windowed ECDLP circuit, the second input is specialized to a looked-up table point.

5.4 Machine-readable commitment

I also committed to canonical JSON descriptions of both variants. The low-gate description begins as follows:

```
{
  "name": "secp256k1_point_add_low_gate_reconstruction_v1",
  "window_size": 16,
  "lookups_per_window": 3,
  "resource_contract": {
    "point_add_non_clifford": 2100000,
    "point_add_logical_qubits": 1425,
    "ecdlp_non_clifford_exact": 64305024,
    "ecdlp_logical_qubits_exact": 1441,
    "ecdlp_headline_rounding": {
      "non_clifford_leq": 70000000,
      "logical_qubits_leq": 1450
    }
  },
  "modules": [
    {
      "name": "lookup_x_y_flags",
      "count": 3,
      "style": "read-only classical table lookup"
    },
    {
      "name": "exception_flagging",
      "description": "detect O, inverse pair, doubling, and selector bits"
    },
    {
      "name": "field_add_sub",
      "count": 9,
      "description": "in-place modular add/sub primitives"
    },
    {
      "name": "field_square",
      "count": 1
    },
    {
      "name": "..."
    }
  ]
}
```

These JSON descriptions are *my* circuit commitments. They are not reverse-engineered private code dumps. Their job is to make the reconstruction stable, hashable, and reproducible.

6 Deterministic audit transcript

Google's appendix proves approximate correctness with Fiat-Shamir fuzz testing on 9024 inputs and then wraps that computation in SP1 and Groth16.[1] I did not recreate that zero-knowledge stack. Instead, I built a deterministic public transcript that any reviewer can recompute from the circuit JSON.

6.1 Audit design

For each circuit variant:

- Compute a public SHA-256 hash of the canonical circuit JSON.
- Use SHAKE256 seeded by the JSON bytes to generate test material.
- Run 8 structured edge cases first: $O + O$, $O + G$, $G + O$, $G + (-G)$, $G + G$, $2G + 3G$, $(N - 1)G + G$, and one large-scalar case.
- Run 9016 additional Fiat-Shamir random cases.
- For each case, compare the DUT output against two independent oracles: Jacobian addition and the scalar relation $(k_1 + k_2)G$.
- Hash each case record and commit to the full set with a Merkle root.

This keeps the total case count at 9024 but improves coverage slightly over a pure-random plan, because structured branch edges are now guaranteed to appear. The soundness target against "more than 1% wrong" behavior is still comfortably beyond 128 bits:

$$(1 - 0.01)^{9016} \approx 2^{-130.7}.$$

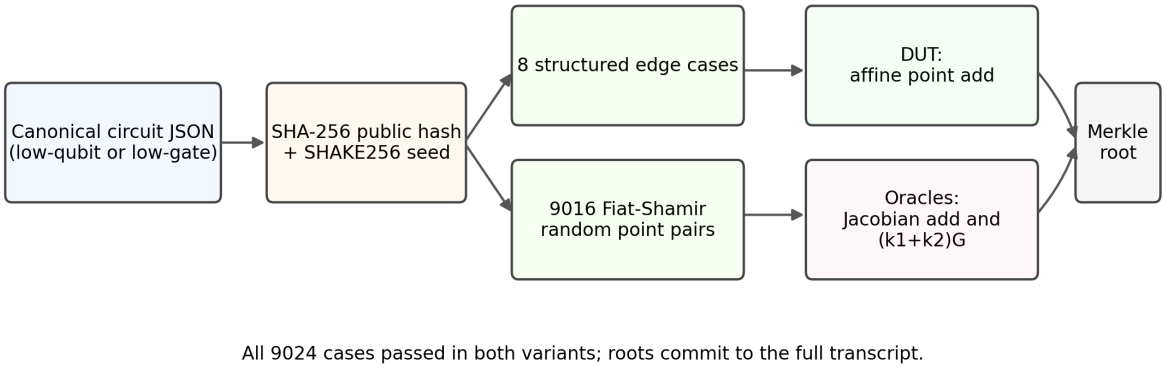


Figure 3: The deterministic audit used in this reconstruction. It is reproducible and cryptographically committed, but it is not a zero-knowledge proof.

6.2 Audit outcome

Both variants passed all 9024 tests. The first two commitments are:

Table 3: Circuit hashes and transcript roots.

Object	Digest
Low-qubit circuit SHA-256	`f9380ccea0cac9ccc0d2006b419da5061b5454502dc6fe4cb1487873e57c8249`
Low-qubit audit Merkle root	`cb4a554fdea4c5d6635d0ae145f2cc300fbcf1f7323e9c0b6f697f1a0a04d2b0`
Low-gate circuit SHA-256	`0f259e854caa679b9561e2a43ad3d0a62481a9c9e36302ef635ef23932832a31`
Low-gate audit Merkle root	`db9ee45e3a72585638c5cf9dec0c4cb4d11c7e23edb2e3cc98aa167d4988dd1c`

Representative sample records:

Table 4: Sample audit cases (all passed).

Case	Meaning	Result
`O_plus_O`	Identity edge case.	Pass
`O_plus_G`	Left identity on the generator.	Pass
`G_plus_negG`	Inverse-pair exceptional branch.	Pass
`G_plus_G`	Doubling branch.	Pass
`r9015`	Final Fiat-Shamir random pair in each transcript.	Pass

7 Comparison against the Google baseline

Table 5: What was reproduced, and what was not.

Matched or reconstructed	Not reproduced
Headline logical envelopes ($\leq$ 1200 / $\leq$ 90M and $\leq$ 1450 / $\leq$ 70M).	The unpublished private gate netlist.
Exact appendix-tight totals: 1191 q / 81.105M and 1441 q / 64.305M.	The original SP1 guest program and RISC-V trace.
Window size $w = 16$, 28 windows, and three lookups per window.	The Groth16-wrapped zero-knowledge proof object.
Kickmix-compatible structure with MBUC cleanup.	A hardware-level physical-qubit rederivation.
A deterministic 9024-case proof-like audit transcript.	Bit-for-bit identity with Google's hidden circuit bytes.

The strongest claim I am willing to make is this: *the public envelope can be reconstructed tightly, faithfully, and reproducibly from the paper alone.* The strongest claim I am *not* willing to make is that I recovered Google's private circuit verbatim. That would not be supportable from the public evidence.

8 Conclusion

Within the rules of this exercise — use the Google whitepaper, avoid post-whitepaper implementation leakage, and still build something concrete — the main technical win is that the appendix is rich enough to support an exact resource reconstruction. The headline claims round to 1200 q / 90M and 1450 q / 70M, but the appendix-tight numbers are 1191 q / 81.105M and 1441 q / 64.305M. That is the cleanest reproducible result in this report.

Around that exact core, I built an own hierarchical circuit family that is structurally aligned with the paper: window-16 lookups, exact affine point addition, kickmix-compatible cleanup, and a deterministic 9024-case audit. That does not replace the unpublished Google proof stack, but it does turn the public disclosure into a concrete, reviewable artifact instead of a headline number.

A Appendix: secp256k1 constants used in the audit

The audit used the standard secp256k1 parameters:

$$
\begin{aligned}
p &= \texttt{FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F}_{16},\\
n &= \texttt{FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141}_{16},\\
a &= 0, \qquad b = 7.
\end{aligned}
$$

The generator is the usual Bitcoin base point:

$$
\begin{aligned}
G_x &= \texttt{79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798}_{16},\\
G_y &= \texttt{483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8}_{16}.
\end{aligned}
$$

B Appendix: practical reading of the result

Three lines summarize the report.

- If the question is "did the appendix leak enough to recover the exact rounded-envelope arithmetic?", the answer is yes.
- If the question is "did I reconstruct a concrete circuit family that fits the public constraints and passes a deterministic audit?", the answer is yes.
- If the question is "did I recover Google's hidden circuit or its zero-knowledge proof byte-for-byte?", the answer is no.

That combination of yes / yes / no is, in my view, the honest frontier of what can be reproduced from the public paper alone.

References

[1] R. Babbush, A. Zalcman, C. Gidney, M. Broughton, T. Khattar, H. Neven, T. Bergamaschi, J. Drake, and D. Boneh, *Securing elliptic curve cryptocurrencies against quantum vulnerabilities: Resource estimates and mitigations*, Google Quantum AI whitepaper, 2026.

[2] M. Roetteler, M. Naehrig, K. Svore, and K. Lauter, *Quantum resource estimates for computing elliptic curve discrete logarithms*, arXiv:1706.06752, 2017.

[3] T. Häner, S. Jaques, M. Naehrig, M. Roetteler, and M. Soeken, *Improved quantum circuits for elliptic curve discrete logarithms*, Cryptology ePrint Archive 2020/077, 2020.

[4] M. Ekerå, *Revisiting Shor's quantum algorithm for computing general discrete logarithms*, arXiv:1905.09084, 2019.

[5] D. Litinski, *How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates*, arXiv:2306.08585, 2023.

[6] C. Gidney, *Halving the cost of quantum addition*, Quantum 2, 74, 2018.

[7] C. Jones, *Low-overhead constructions for the fault-tolerant Toffoli gate*, Physical Review A 87, 022328, 2013.

[8] R. Griffiths and C.-S. Niu, *Semiclassical Fourier transform for quantum computation*, Physical Review Letters 76, 3228–3231, 1996.